# COMPUTER SCIENCE

## PAPER 1 (THEORY)

**Maximum Marks: 70** **Time Allowed: Three Hours**

*Reading Time: Additional Fifteen Minutes*

**Instructions to Candidates**

1. You are allowed an additional fifteen minutes for only reading the question paper.
2. You must NOT start writing during the reading time.
3. This question paper has 13 printed pages.
4. It is divided into two parts: Part I and Part II.
5. It has eleven questions in all.
6. Part I is compulsory and has two questions.
7. While attempting Multiple Choice Questions in Part I, you are required to write only ONE option as the answer.
8. Part II is divided into three sections: A, B and C.
9. Each section in Part II has three questions. Any two questions have to be attempted from each section.
10. The intended marks for questions are given in brackets [ ].

**Instruction to Supervising Examiner**

1. Kindly read aloud the Instructions given above to all the candidates present in the examination hall.

## PART I – 20 MARKS

*Answer all questions.*

*While answering questions in this Part, briefly indicate your working and reasoning, wherever required.*

**Question 1**

(i) Consider the class given below, where show( ) is a static member method: [1]

```
class Student
{
    int roll;
    static int count;
    void greet()
    {
        System.out.println("Hello");
    }
    static void show()
    {
        ...
    }
}
```

Which of the following statements are valid for show( )?

I. It can be invoked as `Student.show( )` without creating any object.

II. It can call the non-static method `greet( )` directly by its name.

III. It cannot refer to `this` or `super`.

*(Application)*

(a) I and III

(b) II and III

(c) I and II

(d) Only III

(ii) According to the Principle of Duality, the Boolean equation P • 1 + P′ • Q + P • Q = P + Q will be equivalent to: *(Application)* [1]

(a) (P + 1) • (P′ + Q) • (P + Q) = P • Q

(b) (P + 0) • (P′ + Q) • (P + Q) = P • Q

(c) (P + 0) • (P′ + Q) • (P + Q) = P + Q

(d) P • 0 + P′ + Q • P + Q = P • Q

(iii) The complement of the Boolean expression (A + B′) • (A′ + C) is: *(Application)* [1]

(a) A • B′ + A′ • C

(b) (A′ + B) • (A + C′)

(c) A′ • B′ + A • C

(d) A′ • B + A • C′

(iv) The canonical expression of F(P, Q, R) = π(1, 4, 6) is: *(Application)* [1]

(a) (P + Q + R′) • (P′ + Q + R) • (P′ + Q′ + R)

(b) (P + Q + R′) • (P′ + Q + R) • (P′ + Q′ + R′)

(c) (P′ + Q′ + R) • (P + Q′ + R′) • (P + Q + R′)

(d) (P + Q′ + R) • (P′ + Q + R′) • (P′ + Q′ + R′)

(v) Given below are two statements marked Assertion and Reason. Read the two statements carefully and choose the correct option. [1]

**Assertion:** An abstract class can contain abstract as well as non-abstract methods.
**Reason:** An abstract class can be instantiated with the `new` operator, so that its non-abstract methods can be used directly.

*(Analysis)*

(a) Both Assertion and Reason are true and Reason is the correct explanation for Assertion.

(b) Both Assertion and Reason are true but Reason is not the correct explanation for Assertion.

(c) Assertion is true and Reason is false.

(d) Both Assertion and Reason are false.

(vi) Study the given propositions and the statements marked Assertion and Reason that follow. Choose the correct option on the basis of your analysis. [1]

P – It is raining. Q – The road is wet.
S1 = P ⇒ Q S2 = ~Q ⇒ ~P

**Assertion:** S2 is the converse of S1.
**Reason:** The converse of a conditional statement is formed by negating both its antecedent and its consequent.

*(Analysis)*

(a) Both Assertion and Reason are true and Reason is the correct explanation for Assertion.

(b) Both Assertion and Reason are true but Reason is not the correct explanation for Assertion.

(c) Assertion is true and Reason is false.

(d) Both Assertion and Reason are false.

(vii) Study the given code: [1]

```
interface Printable
{
    void show();
}
interface Displayable
{
    void show();
}
class Doc implements Printable, Displayable
{
    ...
}
```

How many times must the method `show( )` be defined in the class Doc?

*(Analysis)*

(a) Twice

(b) Once

(c) Compile time error

(d) Run time error

(viii) The worst case time complexity of the following code segment is: [1]

```
for (i = 0; i < p; i++)
    s = s + i;
for (j = 0; j < q; j++)
    t = t * 2;
```

*(Analysis)*

(a) O(p)

(b) O(q)

(c) O(p + q)

(d) O(pq)

(ix) Differentiate between a tautology and a contradiction. *(Understanding)* [1]

(x) What is Gray code in a Karnaugh map? *(Understanding)* [1]

**Question 2**

(i) Convert the following infix notation to postfix form: [2]

((M − N) / (P * (Q + R))) + S ^ (T − U) *(Application)*

(ii) A matrix P[−6 … 11, 2 … 15] is stored in the memory with each element requiring 4 bytes of storage. If the base address of P[−6][2] is 3000, find the memory address of P[4][9] when the matrix is stored column major wise. *(Application)* [2]

(iii) The following function findPos( ) is a part of a class. Assume arr[ ] is a sorted array of positive integers. Answer the questions given below along with dry run / working.

```
public int findPos(int arr[], int f, int l, int x)
{
    if (f > l)
        return -1;

    int m = (2 * f + l) / 3;

    if (arr[m] == x)
        return m;
    else if (arr[m] > x)
        return findPos(arr, f, m - 1, x);
    else
        return findPos(arr, m + 1, l, x);
}
```

(a) Trace the recursive calls and find the output when arr[ ] = {4, 9, 15, 21, 28, 36, 47, 60}, f = 0, l = 7 and x = 47. *(Analysis)* [2]

(b) What additional concept (apart from recursion) is used in the above function logic? *(Analysis)* [1]

(iv) The following is a function of class Tribonacci. This recursive function calculates and returns the $n^{th}$ term of the Tribonacci series, which is similar to the Fibonacci series but each term is the sum of the previous **three** terms. (Example: 0, 1, 1, 2, 4, 7, 13, 24, … for n = 0, 1, 2, …)

Some statements or expressions are missing and are marked by ?1?, ?2? and ?3?. Replace each with suitable code so that the function works correctly.

```
public int tribonacci(int n)
{
    if (?1?)
        return 0;
    else if (?2?)
        return 1;
    else
        return tribonacci(n - 1) + tribonacci(n - 2) + ?3?;
}
```

(a) What condition should replace ?1? *(Analysis)* [1]

(b) What condition should replace ?2? *(Analysis)* [1]

(c) What recursive call or expression should replace ?3? *(Analysis)* [1]

## PART II – 50 MARKS

*Answer six questions in this part, choosing two questions from Section A, two from Section B and two from Section C.*

## SECTION A

*Answer any two questions.*

**Question 3**

(i) A school designs a smart lighting system for its corridors. The system sets the lights to **high power** only when any one of the following conditions is satisfied: [5]

- Motion is not detected in the corridor and the ambient light is low.

  OR
- Motion is detected and the ambient light is low, but the current time is not between 6 PM and 6 AM.

  OR
- Motion is detected and the ambient light is not low, but the manual switch override has been turned on by the caretaker.

The inputs are:

**INPUTS**

| | |
|---|---|
| M | Motion is detected in the corridor. |
| L | The ambient light level is low. |
| T | The current time is between 6 PM and 6 AM. |
| S | The manual switch override is turned on. |

(In all the above cases, 1 indicates yes and 0 indicates no.)
Output: X – 1 indicates high power, 0 indicates low power, for all cases.

Draw the truth table for the inputs and the output given above and write the POS expression for X(M, L, T, S).

*(Application)*

(ii) Reduce the above expression X(M, L, T, S) by using a 4-variable Karnaugh map, showing the various groups (i.e. octal, quads and pairs). Draw the logic gate diagram for the reduced expression. Assume that the variables and their complements are available as inputs. *(Create)* [5]

**Question 4**

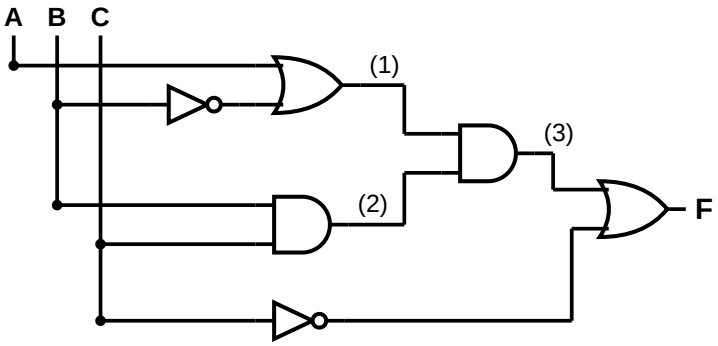
(i)

(a) Given F(A, B, C, D) = (A + B + C + D) • (A + B + C′ + D) • (A + B′ + C + D) • (A + B′ + C′ + D) • (A′ + B + C + D) • (A′ + B + C′ + D) • (A′ + B′ + C + D) • (A′ + B′ + C′ + D) • (A′ + B′ + C′ + D′), reduce the above expression by using a 4-variable Karnaugh map, showing the various groups (i.e. octal, quads and pairs). *(Application)* [4]

(b) Draw the logic gate diagram for the reduced expression. Assume that the variables and their complements are available as inputs. *(Create)* [1]

(ii)

(a) From the logic circuit diagram given below, name the outputs (1), (2) and (3) and finally derive the Boolean expression F and simplify it. State whether F is a tautology, a contradiction or a contingency. *(Application)* [4]

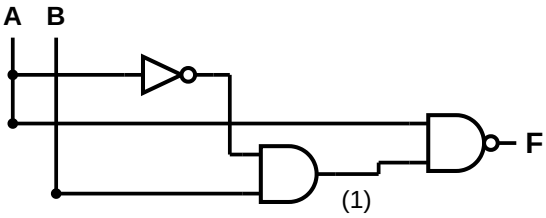



(b) Write the dual of the expression (A + 0) • B. *(Application)* [1]

**Question 5**

(i) Draw the logic circuit to decode the following binary numbers (0010, 0100, 0110, 1001, 1011, 1101, 1110, 1111) to their hexadecimal equivalents. Also state the hexadecimal equivalents of the given binary numbers. *(Create)* [5]

(ii) In the logic gate diagram given below, F is a tautology. State whether the statement is true or false. Justify your answer using a truth table. *(Analysis)* [3]




(iii) Represent the given expression in canonical and cardinal POS form. [2]

H(x, y, z) = x′•y•z′ + x•y′•z′ + x•y•z + x′•y•z *(Application)*

## SECTION B

*Answer any two questions.*

*Each program should be written in such a way that it clearly depicts the logic of the problem. This can be achieved by using mnemonic names and comments in the program. (Flowcharts and Algorithms are not required.)*

*The programs must be written in Java.*

**Question 6**

A fraction is a combination of two positive integers, a numerator and a denominator. A fraction is in its **lowest terms** when its numerator and denominator have no common factor except 1. Design a class Fraction to reduce a fraction to its lowest terms, and to add two fractions. [10]

```
Example 1: 35/77 in lowest terms is 5/11
Example 2: 1/6 + 3/10 = 28/60 = 7/15
```

The details of the members of the class are given below:

| | | |
|---|---|---|
| Class name | : | **Fraction** |
| *Data members/instance variables:* | | |
| n, d | : | integer variables to store the numerator and the denominator respectively |
| *Methods/Member functions:* | | |
| Fraction(int a, int b) | : | parameterised constructor to initialise the data members |
| int fnHCF(int a, int b) | : | to return the highest common factor of a and b using the **recursive technique** |
| Fraction fnReduce(Fraction f1) | : | to reduce the fraction f1 to its lowest terms using fnHCF( ) and to return it as a new object |
| Fraction fnAdd(Fraction f1) | : | to add the current fraction and f1 and to return the sum in its lowest terms, by calling fnReduce( ) |
| void fnShow( ) | : | to display the fraction in the form n/d |

Specify the class Fraction giving details of the constructor( ), int fnHCF(int, int), Fraction fnReduce(Fraction), Fraction fnAdd(Fraction) and void fnShow( ). Define a main( ) function to create objects and call the functions accordingly to enable the task.

*(Create)*

**Question 7**

A sports academy wants to hold a friendly match between two teams of at most fifteen players each. A class Squad stores the name of a team, the number of players and the height of each player in a one-dimensional array. Design the class Squad to compare the average heights of two teams. [10]

The details of the members of the class are given below:

| | | |
|---|---|---|
| Class name | : | **Squad** |
| *Data members/instance variables:* | | |
| ht[ ] | : | to store the height (in cm) of each player |
| tname | : | to store the name of the team |
| n | : | to store the total number of players |
| *Methods/Member functions:* | | |
| Squad( ) | : | default constructor |
| Squad(int num, String nm) | : | to initialise the number of players and the name of the team |
| void fillHeight( ) | : | to enter the heights of all the players |
| double averageHeight( ) | : | to calculate and return the average height of the team |
| String compare(Squad s1, Squad s2) | : | to compare the average heights of the teams s1 and s2 and to return the name of the team with the higher average height; if both averages are equal, to return the word “SAME” |
| void display(String msg) | : | to display the name of the team with the higher average height stored in msg; if msg is “SAME”, to display “Both teams have the same average height.” |

Specify the class Squad giving details of the constructors, void fillHeight( ), double averageHeight( ), String compare(Squad, Squad) and void display(String). Write the main( ) function to call the methods defined in an appropriate manner.

*(Create)*

**Question 8**

Two words are called **anagrams** of each other if both contain the same letters, the same number of times, but arranged in a different order. [10]

Following are three pairs of words which are anagrams of each other:

```
angle, glean
night, thing
stone, tones
```

Design a class Anagram to input two words in lower case and check whether they are anagrams of each other. The details of the members of the class are given below:

| | | |
|---|---|---|
| Class name | : | **Anagram** |
| *Data members/instance variables:* | | |
| s1, s2 | : | to store the two words in lower case |
| l1, l2 | : | to store the number of characters in s1 and s2 respectively |
| *Methods/Member functions:* | | |
| Anagram(String a, String b) | : | parameterised constructor to assign the accepted values to s1 and s2 and their lengths to l1 and l2 |
| void sort(char c[ ]) | : | to sort the characters of the array c[ ] in ascending order using any sorting technique |
| boolean areAnagram(char[ ] str1, char[ ] str2) | : | to check whether str1 and str2 are anagrams of each other and to return true or false accordingly |
| void check( ) | : | to convert the two words into two character arrays and to display the appropriate message, as shown in the sample output, by calling the method areAnagram( ) |

Specify the class Anagram giving details of the parameterised constructor, void sort(char c[ ]), boolean areAnagram(char[ ], char[ ]) and void check( ). Create one object in the main( ) method and call all the methods appropriately.

```
Sample Input:  Enter first word  -> angle
               Enter second word -> glean
Output:        angle and glean are anagram of each other

Sample Input:  Enter first word  -> cat
               Enter second word -> cut
Output:        cat and cut are not anagram of each other
```

*(Create)*

## SECTION C

*Answer any two questions.*

*Each program should be written in such a way that it clearly depicts the logic of the problem stepwise. This can be achieved by using comments in the program and mnemonic names or pseudo codes for algorithms. The programs must be written in Java and the algorithms must be written in general / standard form, wherever required / specified.*

*(Flowcharts are not required.)*

**Question 9**

(i) A clinic issues integer tokens to its patients. A new token can be added only at one end (rear) and a token can be removed only from the other end (front). This forms a restricted linear data structure. The details of the class TokenLine are given below: [3]

| | | |
|---|---|---|
| Class name | : | **TokenLine** |
| *Data members/instance variables:* | | |
| tk[ ] | : | an array to hold a maximum of 100 integer elements |
| cap | : | stores the capacity of the array |
| f | : | to point to the index of the front |
| r | : | to point to the index of the rear |
| *Methods/Member functions:* | | |
| TokenLine(int m) | : | constructor to initialise the data members cap = m, f = 0, r = −1 and to create the integer array tk[ ] |
| void addToken(int v) | : | to insert an element from the rear, if space is available; otherwise display the message “Overflow! Insertion not possible.” |
| int serveToken( ) | : | to remove and return an element from the front, if the structure is not empty; otherwise return −9999 and display “Underflow! Deletion not possible.” |
| void disp( ) | : | to display the current elements from front to rear |

(a) Write the member functions void addToken(int) and int serveToken( ). The other functions have been defined; the main( ) function and algorithm need NOT be written.

*(Create)*

(b) What is the common name of the entity described above? *(Recall)* [1]

(c) On what principle does this entity work? *(Recall)* [1]

**Question 10**

A super class Circle has been defined to calculate the area of a circle. Define a sub class Volume to calculate the volume of a cone whose base is that circle. The details of the members of both the classes are given below: [5]

Class name : **Circle**

*Data members/instance variables:*

radius : to store the radius in decimals

area : to store the area of a circle

*Methods/Member functions:*

Circle(…) : parameterised constructor to assign values to the data members

void cal_area( ) : to calculate the area of the circle ($\pi r^2$)

void display( ) : to display the area of the circle

Class name : **Volume**

*Data members/instance variables:*

height : to store the height of the cone in decimals

volume : to store the volume of the cone in decimals

*Methods/Member functions:*

Volume(…) : parameterised constructor to assign values to the data members of both the classes

double calculate( ) : to calculate and return the volume of the cone using the formula $(1/3)\pi r^2 h$, i.e. one third of the area of the base multiplied by the height h

void display( ) : to display the area of the circle and the volume of the cone

Assume that the super class Circle has been defined. Using the concept of inheritance, specify the class Volume giving the details of the constructor(…), double calculate( ) and void display( ).

The super class, main function and algorithm need NOT be written.

*(Create)*

**Question 11**

(i) A linked list is formed from the objects of the class Node. The class structure of the Node is given below: [2]

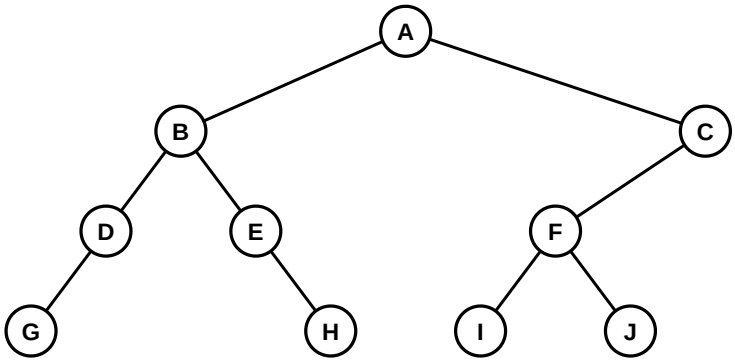
```
class Node
{
    int num;
    Node next;
}
```

Write an Algorithm OR a Method to delete the node at the $n^{th}$ position (the first node is at position 1) of an existing linked list, and return the reference to the first node of the list. If the list has fewer than n nodes it is left unchanged. The method declaration is as follows:

```
Node deleteNth(Node start, int n)
```

*(Analysis)*

(ii) Answer the following questions from the diagram of a Binary Tree given below. (The root is at level 0; the height of a leaf is 0.)




(a) Write the post-order traversal of the tree. *(Understanding)* [1]

(b) State the height of node C and the level of node I. *(Understanding)* [1]

(c) State the size of the left sub tree and the degree of node F. *(Understanding)* [1]

— — — —